Epic 4: Model Building

Story 2: Random Forest Model

The Random Forest algorithm is implemented to improve the accuracy and robustness of flood prediction by combining the outputs of multiple decision trees. A reusable function named randomForest() is created to train, test, and evaluate the model using the training and testing datasets.

Random Forest Algorithm

The RandomForestClassifier from the Scikit-learn library is used to build the model. Random Forest is an ensemble learning algorithm that constructs multiple decision trees during training and combines their predictions through majority voting. This approach reduces overfitting and improves the model's generalization performance.

Model Initialization

The classifier is initialized with suitable hyperparameters such as:

- **n_estimators – Specifies the number of decision trees in the forest.**
- **random_state – Ensures reproducible and consistent results during model training.**

Model Training

The model is trained using the fit() method with the training dataset (X_train and y_train). During training, each decision tree learns different patterns from randomly selected subsets of the data, making the overall model more accurate and reliable.

Prediction

After training, the predict() method is applied to the testing dataset (X_test) to generate flood prediction results. The predicted values are then compared with the actual target values (y_test) to evaluate the model's performance.

Model Evaluation

The Random Forest model is evaluated using the following performance metrics:

- **Accuracy Score – Measures the percentage of correctly classified instances.**
- **Confusion Matrix – Displays True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).**
- **Classification Report – Provides detailed evaluation metrics including Precision, Recall, F1-Score, and Support.**

Outcome

- **Successfully trained the Random Forest classifier.**

- **Generated predictions using the testing dataset.**
- **Evaluated the model using Accuracy, Confusion Matrix, Precision, Recall, and F1-Score.**
- **Compared the Random Forest model with other machine learning algorithms to identify the best-performing model for flood prediction.**

Figure 13: Random Forest model training and performance evaluation using the flood prediction dataset

```python
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

def randomForest(X_train, X_test, y_train, y_test, n_estimators=100, random_state=42):
    print("\n========== RANDOM FOREST MODEL BUILDING ==========")

    # 1. Initialize Random Forest Classifier
    model = RandomForestClassifier(n_estimators=n_estimators, random_state=random_state)
    print(f"[INFO] RandomForestClassifier initialized with n_estimators={n_estimators}, random_state={random_state}")

    # 2. Train the model
    model.fit(X_train, y_train)
    print("[INFO] Model training completed.")

    # 3. Predict on test data
    y_pred = model.predict(X_test)
    print("[INFO] Prediction completed on test data.")

    # 4. Evaluate the model
    accuracy = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)
    cr = classification_report(y_test, y_pred)

    # 5. Display results
    print(f"\n[RESULT] Accuracy : {accuracy:.4f}")
    print("\nConfusion Matrix:"")
    print(cm)
    print("\nClassification Report:\n")
    print(cr)

    # 6. Return the model and predictions
    return model, y_pred

# ----------------- Example Usage -----------------
# model_rf, y_pred_rf = randomForest(X_train, X_test, y_train, y_test)
```